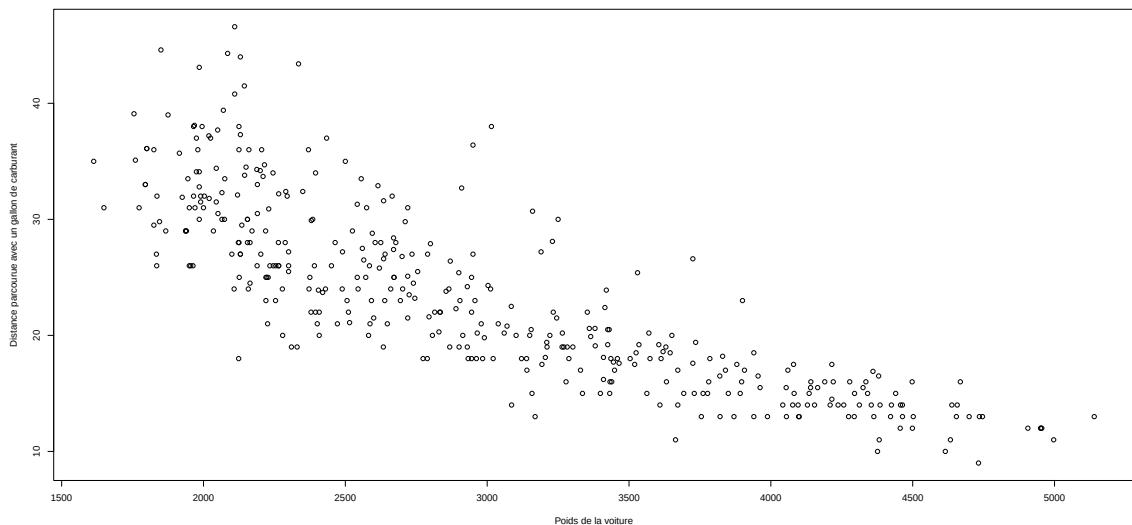


Chapitre 9: Régression non-linéaire

Considérons le jeu de données `Auto` de la library `ISLR` du logiciel R. Ce jeu de données contient 392 observations. La variable réponse est `mpg`, la distance moyenne en miles parcourue par la voiture avec un gallon de carburant. On s'intéresse à une seule variable explicative, soit le poids de la voiture `weight`. On commence par tracer le nuage de points $\{(X_i, Y_i), i = 1, \dots, 392\}$, où X_i et Y_i sont, respectivement, le poids et la distance moyenne en miles parcourue avec un gallon de carburant de la voiture i .

```
> if (!require("ISLR")) install.packages("ISLR")
> library("ISLR")#### Installer la librairie ISLR
>
> data(Auto)
> X <- Auto$weight
> Y <- Auto$mpg
>
> plot(X,Y,xlab="Poids de la voiture",
+      ylab="Distance parcourue avec un gallon de carburant")
```

Et on obtient



Ce graphique montre que la relation entre X et Y n'est clairement pas linéaire. Dans le chapitre précédent, on a considéré deux

approches pour étudier la relation entre X et Y ; soient l'estimateur des k plus proches voisins et l'estimateur par noyau. Ces approches nous ont permis d'estimer $f(x) = E[Y|X = x]$. Dans ce chapitre, on présente des approches alternatives pour estimer $f(x)$.

1 Régression polynomiale

L'idée de la régression polynomiale est d'approximer $f(x)$ par un polynôme de degré p . Ainsi, on pose

$$f(x) = \beta_0 + \beta_1 x + \cdots + \beta_p x^p$$

et on obtient le modèle linéaire

$$Y_i = \beta_0 + \beta_1 X_i + \cdots + \beta_p X_i^p + \epsilon_i, \quad i = 1, \dots, n.$$

On peut faire l'hypothèse que $\epsilon_i \sim \mathcal{N}(0, \sigma^2)$ ou que simplement $E[\epsilon_i] = 0$ et $Var[\epsilon_i] = \sigma^2 > 0$.

Dans tous les cas, on estime $\beta = (\beta_0, \beta_1, \dots, \beta_p)^\top$ par

$$\hat{\beta} = (X^\top X)^{-1} X^\top Y,$$

où

$$X = \begin{pmatrix} 1 & X_1 & \cdots & X_1^p \\ 1 & X_2 & \cdots & X_2^p \\ \vdots & \vdots & \ddots & \vdots \\ 1 & X_n & \cdots & X_n^p \end{pmatrix}.$$

Le vecteur β de coefficients de régression contient $p + 1$ éléments. On dit qu'on a $p + 1$ degrés de liberté.

Pour sélectionner la valeur la plus adéquate pour p , on peut se fier à des critères comme l'AIC ou le BIC ou encore effectuer des tests d'hypothèses avec des statistiques F . Dans ce chapitre, on adopte l'esprit du chapitre précédent et on choisit la valeur de p qui minimise le taux d'erreur test. On peut estimer le taux d'erreur test avec la procédure qui exclut une observation du jeu de données entraînement à la fois. On a vu au Chapitre 7 que cette méthode nous conduit à une expression explicite pour le critère de la validation croisée.

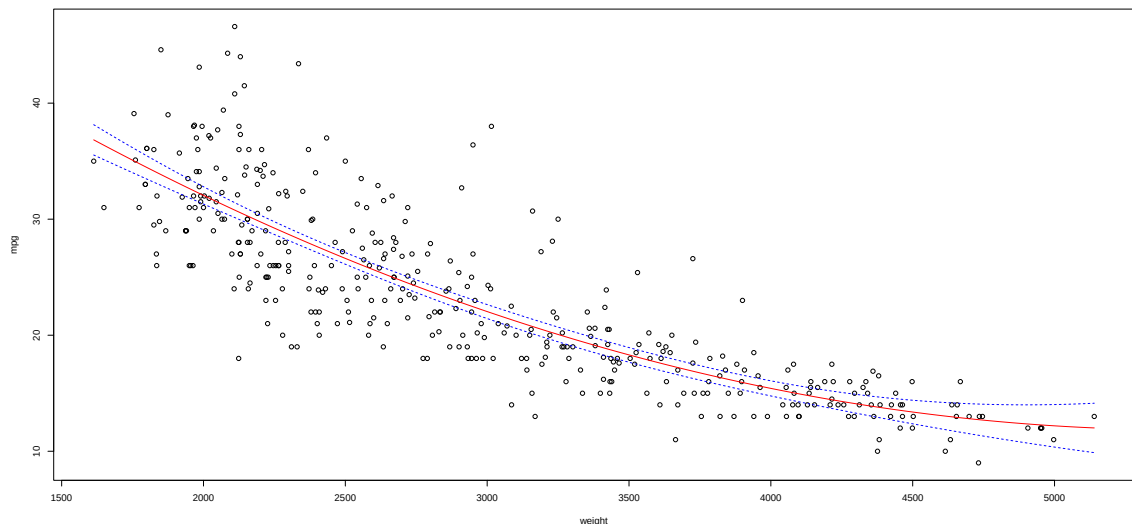
Une fois qu'on choisit la valeur de p , on a alors

$$\hat{f}(x) = \hat{\beta}_0 + \hat{\beta}_1 x + \cdots + \hat{\beta}_p x^p = x_*^\top \hat{\beta};$$

où $x_* = (1 \ x \ x^2 \ \cdots \ x_p)$. On en déduit que $Var[\hat{f}(x)] = x_* Var[\hat{\beta}] x_*^\top$. Cette expression nous permet d'obtenir des bandes de confiance autour de $\hat{f}(x)$.

```
> #####
> ## Regression polynomiale ##
> #####
>
> modele.polynomial <- lm(mpg~ poly(weight,2,raw=TRUE),data=Auto)
>
> axe.poids <- seq(min(Auto$weight),max(Auto$weight),1)
> prediction.polynomial <- predict(modele.polynomial,
>                                 newdata=list(weight=axe.poids),se=TRUE)
>
> plot(mpg~weight,data=Auto)
> lines(axe.poids,prediction.polynomial$fit,col="red")
> lines(axe.poids,
> prediction.polynomial$fit-1.96*prediction.polynomial$se.fit,lty=2,col="blue")
> lines(axe.poids,
> prediction.polynomial$fit+1.96*prediction.polynomial$se.fit,lty=2,col="blue")
```

Et on obtient



La régression polynomiale peut être vue comme une régression sur des transformations connues de la variable explicative:

$$Y_i = \beta_0 b_0(X_i) + \beta_1 b_1(X_i) + \cdots + \beta_p b_p(X_i) + \epsilon_i, \quad i = 1, \dots, n,$$

avec $b_k(x) = x^k$ pour $k = 0, 1, \dots, p$. Cette approche peut être généralisée de plusieurs façons.

2 Régression avec des fonctions constantes par morceaux

Posons $[A, B]$ le domaine de définition de la variable explicative X . Dans la pratique, on peut par exemple, poser $A = \min\{X_1, \dots, X_n\} - \varepsilon$ et $B = \max\{X_1, \dots, X_n\} + \varepsilon$ où ε est un très petit réel positif.

Considérons une partition $A = c_0 < c_1 < \dots < c_K = B$ de l'intervalle $[A, B]$. Pour $k = 1, \dots, K$, on définit la fonction indicatrice $b_k(x)$ par:

$$b_k(x) = \begin{cases} 1 & \text{si } x \in [c_{k-1}, c_k] \\ 0 & \text{sinon} \end{cases}$$

Cette approche approxime $f(x)$ par une fonction constante par escalier: $f(x) = \beta_k$ si $x \in [c_{k-1}, c_k]$. Autrement dit, on a

$$f(x) = \beta_1 b_1(x) + \beta_2 b_2(x) + \cdots + \beta_K b_K(x)$$

et le modèle devient

$$Y_i = \beta_1 b_1(X_i) + \beta_2 b_2(X_i) + \cdots + \beta_K b_K(X_i) + \epsilon_i$$

La variable explicative X a donc été discrétisée et on a un modèle d'ANOVA à un facteur fixe à K modalités. On montre que l'estimateur des moindres carrés de β_k est

$$\hat{\beta}_k = \frac{\sum_{i=1}^n b_k(X_i) Y_i}{n_k},$$

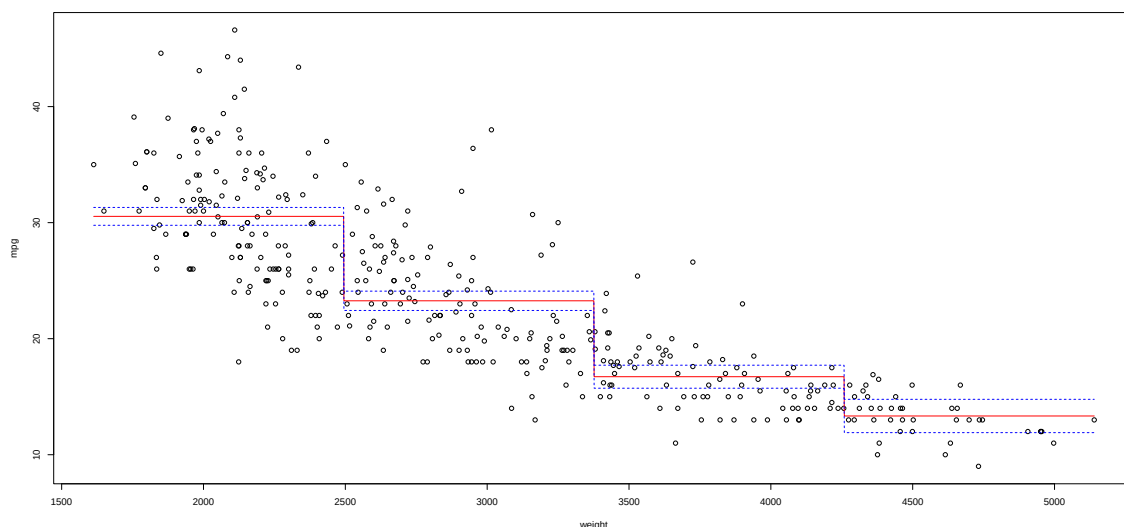
où $n_k = \sum_{i=1}^n b_k(X_i)$ est le nombre d'observations X_i qui appartiennent à $[c_{k-1}, c_k]$. Finalement, on a $\text{Var}[\hat{\beta}_k] = \sigma^2/n_k$.

```

> #####
> ## Regression constante par morceaux ##
> #####
>
> modele.par.morceaux <- lm(mpg~ cut(weight,4),data=Auto)
> prediction.par.morceaux <- predict(modele.par.morceaux,
>                                   newdata=list(weight=axe.poids),se=TRUE)
>
> plot(mpg~weight,data=Auto)
> lines(axe.poids,prediction.par.morceaux$fit,col="red")
> lines(axe.poids,
> prediction.par.morceaux$fit-1.96*prediction.par.morceaux$se.fit,lty=2,col="blue")
> lines(axe.poids,
> prediction.par.morceaux$fit+1.96*prediction.par.morceaux$se.fit,lty=2,col="blue")

```

Et on obtient



3 Régression avec des splines

Les splines sont des polynômes définis par intervalles. On commence par un cas simple avec $K = 2$ et $p = 3$. On divise le domaine $[A, B]$ en deux

intervalles : $[c_0, c_1)$ et $[c_1, c_2)$. On approxime la fonction f par

$$f(x) = \begin{cases} \beta_0 + \beta_1 x + \beta_2 x^2 + \beta_3 x^3 & \text{si } x \in [c_0, c_1), \\ \gamma_0 + \gamma_1 x + \gamma_2 x^2 + \gamma_3 x^3 & \text{si } x \in [c_1, c_2). \end{cases}$$

On a donc $4 + 4 = 8$ degrés de liberté. En plus, on impose que f soit continue et deux fois dérivable au point c_1 . Ce qui nous donne 3 contraintes. Finalement, on se retrouve avec $8-3=5$ degrés de liberté.

Généralisons ce qu'on vient de faire à K et p arbitraires. On a K polynômes de degré p , ce qui nous donne $K \times (p + 1)$ paramètres. Aux $K - 1$ points intérieurs, on impose que f soit continue et $p - 1$ fois dérivable, ce qui nous donne $p \times (K - 1)$ contraintes. Finalement, on a $K(p + 1) - (K - 1)p = K + p$ degrés de liberté.

Pour un p et un K donnés, un choix mathématiquement convivial pour construire une base de $K + p$ splines est donné par: $b_1(x) = 1, b_2(x) = x, \dots, b_{p+1}(x) = x^p, b_{p+2}(x) = h(x, c_1), b_{p+3}(x) = h(x, c_2), \dots, b_{K+p}(x) = h(x, c_{K-1})$, où

$$h(x, c) = \begin{cases} (x - c)^p & \text{si } x \geq c \\ 0 & \text{sinon.} \end{cases}$$

Il existe une infinité de choix pour construire une base de $K + p$ splines. En effet, n'importe quelles $K + p$ combinaisons linéaires indépendantes de la base qu'on vient de définir est encore une base de $K + p$ splines.

On définit alors le modèle

$$Y_i = \beta_1 b_1(X_i) + \beta_2 b_2(X_i) + \dots + \beta_{K+p} b_{K+p}(X_i) + \epsilon_i, \quad i = 1, \dots, n.$$

L'estimateur des moindres carrés de $\beta = (\beta_1, \dots, \beta_{K+p})^\top$ est $(X^\top X)^{-1} X^\top Y$, où

$$X = \begin{pmatrix} b_1(X_1) & b_2(X_1) & \dots & b_{K+p}(X_1) \\ b_1(X_2) & b_2(X_2) & \dots & b_{K+p}(X_2) \\ \vdots & \vdots & \ddots & \vdots \\ b_1(X_n) & b_2(X_n) & \dots & b_{K+p}(X_n) \end{pmatrix}.$$

On obtient finalement $\hat{f}(x) = \hat{\beta}_1 b_1(x) + \hat{\beta}_2 b_2(x) + \dots + \hat{\beta}_{K+p} b_{K+p}(x)$.

```
#####
## Splines cubiques ##
#####
```

```
if (!require("splines")) install.packages("splines")
```

```

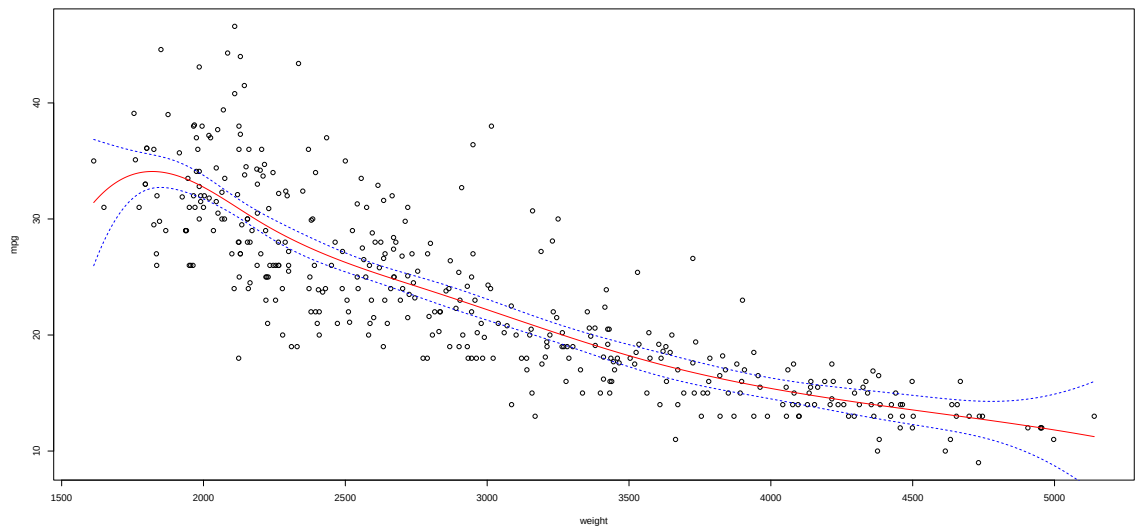
library("splines") ##### Installer la librairie splines

modele.splines <- lm(mpg~bs(weight,knots=c(2200,2800,3600)),data=Auto)
prediction.splines <- predict(modele.splines,
                             newdata=list(weight=axe.poids),se=TRUE)

plot(mpg~weight,data=Auto)
lines(axe.poids,prediction.splines$fit,col="red")
lines(axe.poids,
      prediction.splines$fit+1.96*prediction.splines$se.fit,lty=2,col="blue")
lines(axe.poids,
      prediction.splines$fit-1.96*prediction.splines$se.fit,lty=2,col="blue")

```

Et on obtient



Cet estimateur peut être instable aux extrémités du domaine de X . Pour remédier à ce problème, on considère les splines naturelles. Celles-ci sont définies exactement comme les splines, à l'exception qu'on considère seulement des fonctions linéaires sur les intervalles $[c_0, c_1)$ et $[c_{K-1}, c_K)$.

```

#####
## Splines cubiques naturelles ##
#####

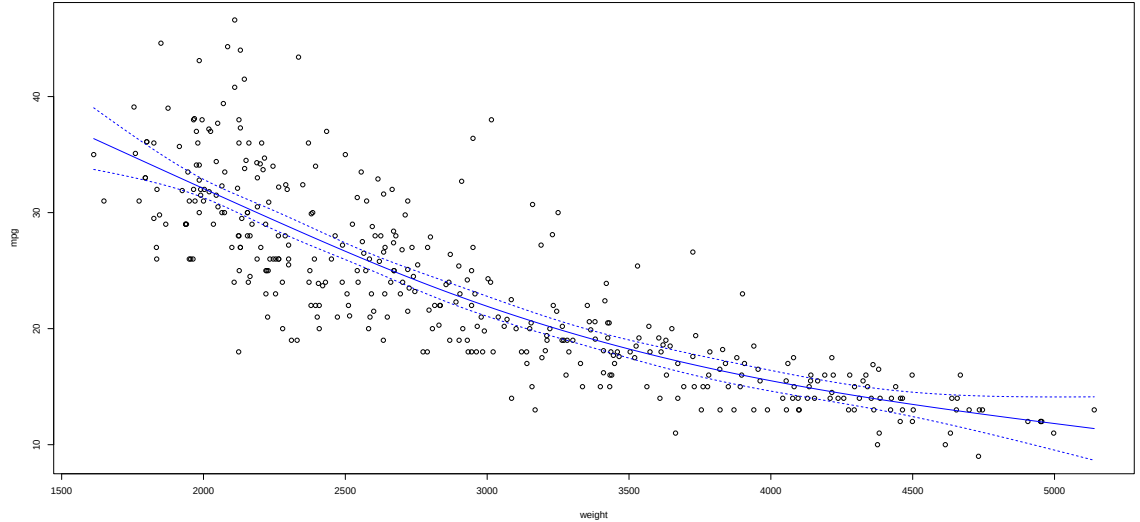
```

```

modele.sn <- lm(mpg~ns(weight,df=4),data=Auto)
prediction.sn <- predict(modele.sn,newdata=list(weight=axe.poids),se=TRUE)
plot(mpg~weight,data=Auto)
lines(axe.poids,prediction.sn$fit,col="blue")
lines(axe.poids,
prediction.sn$fit+1.96*prediction.sn$se.fit,lty=2,col="blue")
lines(axe.poids,
prediction.sn$fit-1.96*prediction.sn$se.fit,lty=2,col="blue")

```

Et on obtient



Un autre défaut des splines est qu'elles peuvent avoir tendance à aller chercher le maximum de points. Pour remédier à ce problème, on impose une pénalité pour les splines trop variables. Ainsi, on estime β en minimisant

$$\sum_{i=1}^n \{Y_i - [\beta_1 b_1(X_i) + \beta_2 b_2(X_i) + \dots + \beta_{K+p} b_{K+p}(X_i)]\}^2 + \lambda \int_A^B f''(t)^2 dt.$$

On montre que l'estimateur qui en résulte est donné par

$$\hat{\beta}_\lambda = (X^\top X + \lambda M)^{-1} X^\top Y.$$

Ici, M est une matrice carrée de dimension $K + p$ et l'élément à la position (l, q) de la matrice M est égal à $\int_A^B b_l''(t) b_q''(t) dt$.

Le paramètre λ est un paramètre qui contrôle la régularité de $\hat{f}$. Dans la pratique, on estime ce paramètre par validation croisée.

```
> #####  
> ## Splines penalites ##  
> #####  
>  
> modele.sp.df.16 <- smooth.spline(Auto$weight,Auto$mpg,df=16)  
>  
> plot(mpg~weight,data=Auto)  
> lines(modele.sp.df.16,col="green")  
>  
> modele.sp.cv <- smooth.spline(Auto$weight,Auto$mpg,cv=TRUE)  
Message d'avis :  
Dans smooth.spline(Auto$weight, Auto$mpg, cv = TRUE) :  
  la validation croise avec des valeurs de 'x' non uniques parat douteuse  
> modele.sp.cv$df  
[1] 10.35561  
> lines(modele.sp.cv,col="purple")
```

Et on obtient

